

CAMPUS EXPENSE & BUDGET MANAGER

Programming in Java

VITyarthi – Build Your Own Project

Student: _____

Registration No.: _____

Date: _____

2. Introduction

Campus Expense & Budget Manager is a terminal-based Java application designed to help students record and analyse everyday expenses. The application demonstrates object-oriented programming, collections, file handling, exception handling, streams, and modular software design.

3. Problem Statement

Students frequently make small payments for food, travel, study materials, subscriptions, and other campus needs. Manual tracking makes it difficult to understand monthly spending and identify whether spending is within a planned allowance. The project addresses this problem using a lightweight command-line Java system.

4. Functional Requirements

1. Expense Management – add, view, and delete expenses.
2. Budget Management – set a monthly budget and monitor spending against it.
3. Reporting & Analytics – calculate monthly total, category-wise totals, transaction count, and highest expense.
4. Persistent Storage – store and reload expense records using CSV.
5. Validation – validate dates, amounts, IDs, and required fields.

5. Non-Functional Requirements

Usability: Menu-driven commands and clear prompts.

Reliability: Invalid input is rejected safely.

Maintainability: Responsibilities are separated into focused classes.

Performance: In-memory collection processing is responsive for typical student-scale datasets.

Resource Efficiency: No external server or database is required.

Error Handling: Application-specific exceptions and I/O handling provide controlled messages.

6. System Architecture

The application follows a simple layered structure: presentation (`Main``), application logic (`ExpenseManager``), domain models (`Expense``, `Budget``), persistence (`ExpenseRepository``), analytics (`ReportService``), and CSV storage.

7. Design Diagrams

The repository contains architecture, workflow, use-case, class, sequence, and ER diagram source files in `docs/diagrams/``. They describe how the user interacts with the CLI and how the Java classes cooperate.

8. Design Decisions & Rationale

`ArrayList`` stores expense records because the primary operations are sequential traversal and filtering.

`HashMap`` associates monthly budgets with months. Java Streams and collectors simplify aggregation. CSV storage keeps the application dependency-free and easy to execute from a terminal. Separation of responsibilities improves maintainability.

9. Implementation Details

The application loads saved expenses at startup. `Main`` displays the menu and forwards operations to `ExpenseManager``. `Expense`` encapsulates one transaction, `Budget`` represents a monthly limit, `ExpenseRepository`` performs CSV I/O, and `ReportService`` performs calculations. Input validation is handled before records are created.

10. Screenshots / Results

The final repository is ready for terminal execution. For the portal report, capture screenshots of the main menu, adding an expense, viewing expenses, budget status, monthly report, invalid input handling, and the `ALL TESTS PASSED` result.

11. Testing Approach

Testing includes manual CLI tests, validation tests, persistence testing, and automated logic checks. The included test class verifies overall total, monthly total, category aggregation, transaction count, and highest expense.

12. Challenges Faced

The key implementation challenges were keeping the application dependency-free, maintaining persistent CSV data, handling malformed input without crashing, and separating responsibilities into meaningful Java classes.

13. Learnings & Key Takeaways

The project provided practical experience with Java classes, constructors, encapsulation, collections, maps, streams, lambda expressions, file I/O, date/time APIs, exception handling, modularity, and testing.

14. Future Enhancements

Possible extensions include authentication, SQLite/MySQL integration, JavaFX GUI, PDF reports, recurring expenses, savings goals, charts, and multiple user profiles.

15. References

Oracle Java SE API documentation; Java documentation for Collections, Streams, `java.time`, and NIO file APIs; VITyarthi Build Your Own Project – General Project Instructions & Submission Guidelines.